

Cardiotox Backend — Frontend Integration Guide

Everything you need to wire the **Cardiotox** frontend to the backend: authentication (email + Google), the AI prediction endpoints, error handling, and the gotchas that will otherwise cost you an afternoon.

Backend base URL (production): `https://cardiotox-backend.onrender.com`

0. TL;DR (read this even if you read nothing else)

1. **Every** request to the backend must include `credentials: 'include'`. Auth here is **cookie-based**, not token-based. If you forget this, login will *look* like it worked but the user will appear logged out on the next call.
 2. There is **no token to store**. Don't touch `localStorage`. The browser holds an HttpOnly session cookie automatically once the user logs in.
 3. To check "is the user logged in?", call `GET /auth/me` → `200` = yes, `401` = no.
 4. **Google login is a full-page redirect**, not a `fetch()`. Send the browser to `.../auth/google` via `window.location.href`.
 5. The AI endpoints (`/api/predict`, `/api/explain`) require the user to be **logged in** and take the **same** `{"data": [...11 numbers...]}` shape you already use against the Hugging Face Space. You point at the backend now instead of HF — the backend does the Gradio two-step behind the scenes and returns the same result shape.
-

1. ⚠️ What I need FROM YOU first (coordination)

Two things depend on **your** frontend URL, and I have to set them on the backend or things break:

1. **Your frontend origin(s)** — I add these to the backend's CORS allowlist (`FRONTEND_ORIGIN`). Cross-origin requests from any origin I haven't whitelisted are **blocked by the browser**. Send me:
 - your local dev origin (e.g. `http://localhost:3000`)
 - your deployed origin (e.g. `https://cardiotox.vercel.app`)
2. **Your single "post-login" URL** (`FRONTEND_URL`) — where the backend redirects users after email verification and after Google login. Usually your deployed root or `/login`.

Until I've added your exact origin, your `fetch()` calls will fail with a CORS error even though the backend is fine. So ping me your URLs before you start testing.

2. How auth works here (the mental model)

This backend uses **server-side sessions with cookies** (via `axum-login` + Postgres), **not JWT**. Practically, for you:

- When the user logs in, the backend sends back a `Set-Cookie` header with an **HttpOnly** session cookie. The browser stores it automatically.
- On every subsequent request, the browser **automatically attaches** that cookie — *but only if you set* `credentials: 'include'` on the request. This is the #1 thing people miss.
- The cookie is **HttpOnly**, so **JavaScript cannot read it** (`document.cookie` won't show it). That's intentional and good for security. You never need to read it — the browser handles it.
- The cookie is `SameSite=None; Secure` in production, which is why **everything must be over HTTPS** (both your Vercel site and the Render backend are, so you're fine).

Consequence: you don't manage tokens, headers, or refresh logic. You just make sure cookies ride along (`credentials: 'include'`), and you ask `/auth/me` when you need to know who's logged in.

3. Setup

Put the base URL in an env var:

```
bash
# .env.local
NEXT_PUBLIC_API_BASE_URL=https://cardiotox-backend.onrender.com
```

A reusable fetch helper (use this everywhere)

```
js
```

```
// lib/api.js
const BASE = process.env.NEXT_PUBLIC_API_BASE_URL;

export async function api(path, { method = "GET", body } = {}) {
  const res = await fetch(`${BASE}${path}`, {
    method,
    credentials: "include", // <-- THE GOLDEN RULE. Always.
    headers: body ? { "Content-Type": "application/json" } : {},
    body: body ? JSON.stringify(body) : undefined,
  });

  // 204 No Content (e.g. logout) has no body
  const data = res.status === 204 ? null : await res.json().catch(() => null);

  if (!res.ok) {
    // Throw an error object the UI can branch on
    throw { status: res.status, data };
  }
  return data;
}
```

Every example below uses this `api()` helper, so `credentials: 'include'` is baked in.

4. Auth endpoints

All bodies are JSON. All state-changing routes are `POST`.

Method	Path	Body	Success	Notes
POST	<code>/auth/register</code>	<code>{email, password, display_name}</code>	<code>201</code>	Sends a verification email. <code>409</code> if email taken.
GET	<code>/auth/verify?token=...</code>	—	redirect	User clicks the email link; backend redirects to <code>FRONTEND_URL/login?verified=1</code> . You don't call this from JS.
POST	<code>/auth/login</code>	<code>{email, password}</code>	<code>200</code> + cookie	<code>403</code> if email not verified, <code>401</code> if wrong credentials.
POST	<code>/auth/logout</code>	—	<code>200/204</code>	Destroys the session server-side.
POST	<code>/auth/password/forgot</code>	<code>{email}</code>	<code>200</code> (always)	Emails a reset link if the account exists. Never reveals whether it does.
POST	<code>/auth/password/reset</code>	<code>{token, new_password}</code>	<code>200</code>	Token comes from the reset email link.
GET	<code>/auth/me</code>	—	<code>200</code> profile / <code>401</code>	Your "am I logged in?" check.

`/auth/me` returns:

```
json
{ "id": "uuid", "email": "user@example.com", "email_verified": true, "display_n
```

Examples

```
js
```

```

import { api } from "@lib/api";

// Register
await api("/auth/register", {
  method: "POST",
  body: { email, password, display_name },
});
// -> 201. Tell the user to check their email to verify.

// Login
await api("/auth/login", { method: "POST", body: { email, password } });
// -> 200, session cookie now set. User is logged in.

// Who am I? (use to gate pages / show user state)
try {
  const me = await api("/auth/me"); // 200 -> logged in
  // show dashboard, me.email, etc.
} catch (e) {
  if (e.status === 401) { /* not logged in -> redirect to /login */ }
}

// Logout
await api("/auth/logout", { method: "POST" });

// Forgot password
await api("/auth/password/forgot", { method: "POST", body: { email } });
// -> always 200. Show "if that email exists, we sent a link".

// Reset password (from the /reset-password page, token in the URL)
await api("/auth/password/reset", { method: "POST", body: { token, new_password } });

```

5. Pages YOU need to build

The backend emails links and redirects that point at **your** frontend. So you own these pages:

1. `/login` — email+password form. Also read `?verified=1` (show a "email verified, please log in" banner) and `?login=success` (Google just succeeded → call `/auth/me` and route to the dashboard).
2. `/register` — signup form. After success, show "check your email".
3. `/reset-password` — reads `?token=...` from the URL, shows a "new password" form, and `POSTS {token, new_password}` to `/auth/password/reset`.
4. `/forgot-password` (optional) — an email field that calls `/auth/password/forgot`.

The verification link and the reset link are generated by the backend using your `FRONTEND_URL`, so make sure the paths above match what I configure.

6. Google OAuth (different — it's a redirect, not a fetch)

Do **NOT** call `/auth/google` with `fetch()`. It's a browser redirect flow. Send the whole page there:

```
jsx
function GoogleLoginButton() {
  const BASE = process.env.NEXT_PUBLIC_API_BASE_URL;
  return (
    <button onClick={() => { window.location.href = `${BASE}/auth/google`; }}>
      Sign in with Google
    </button>
  );
}
```

What happens:

1. Browser goes to `.../auth/google` → backend redirects to Google.
2. User picks their account / consents.
3. Google redirects back to the backend callback → backend creates the session cookie.
4. Backend redirects the browser to `FRONTEND_URL/?login=success`.
5. On that landing, call `/auth/me` to load the user and route them into the app.

Account linking is automatic: if a user signed up with email `x@gmail.com` and later uses "Sign in with Google" with the same `x@gmail.com`, it links to the **same** account (no duplicate). You don't need to do anything for this.

7. AI prediction endpoints

Both are **protected** (user must be logged in → else `401`) and **rate-limited** (→ `429`). The request/response shape is **identical to the Hugging Face Space** you already integrated against, so your existing parsing mostly carries over — you just:

- point at the backend instead of HF,
- **delete** the Gradio two-step / `@gradio/client` code (the backend does that now — you make **one** call and get the result),
- add `credentials: 'include'` (the `api()` helper does this).

Send exactly 11 unscaled numbers, in this order (do **not** scale on the client — the model scales server-side):

```
[ qNet, dwdtmax, vmax, vrest, APD50, APD90, max_dv, camax, carest, CaTD50, CaTD90 ]
```

`POST /api/predict` — risk tier (instant)

```
js
const result = await api("/api/predict", {
  method: "POST",
  body: { data: [0.07, 12.3, 40.1, -88.0, 210.0, 330.0, 8.2, 0.0004, 0.0001, 19
// result is the Gradio "data" array:
//   result[0] = { label: "High", confidences: [ {label:"high",confidence:0.64}
//   result[1] = a human-readable tier string
```

How to read the tier (do this robustly): prefer `result[0]` — it's the clean, structured source.

```
js
const label = result[0]; // { label, confidences: [...] }
const tier = label.label; // "High" | "Intermediate" | "Low"
const confidences = label.confidences; // [{label, confidence}, ...]
```

⚠ `result[1]` is a display string and **may** come back with a prefix like `"Predicted Risk Tier: High"` depending on the model output. Don't rely on it for logic — use `result[0].label` (or the highest-confidence entry in `result[0].confidences`) to determine the tier. Treat `result[1]` as optional display text only.

Note: this is a tree ensemble, so confidences often come back as hard `1.0 / 0.0`. Show it as a **tier**, not as a calibrated clinical probability.

`POST /api/explain` — SHAP explanation (slower, on-demand)

Same 11-value input. Call this only when the user asks (e.g. an "Explain" button) — it takes a few seconds, and the first call after the backend/Space wakes is the slowest.

```
js
```

```
const exp = await api("/api/explain", { method: "POST", body: { data: [...11 nu
const shap = exp[0];
// shap = {
//   predicted_class: "high",
//   base_value: 0.3347,
//   contributions: [ { biomarker: "qNet", value: 43.64, shap: 0.3008 }, ... 11
// }
```

Rendering the SHAP chart (horizontal bars):

- One bar per biomarker; sort by `|shap|` descending.
- Bar length = `shap`; positive = pushed toward the predicted tier (one color), negative = pushed away (another color).
- Row label = `biomarker` + its raw `value`.
- Optional header: "Explaining why this drug was classified `{predicted_class}`", and `base_value + Σ shap ≈ P(predicted tier)`.

8. Error handling cheat sheet

The `api()` helper throws `{ status, data }`. Branch on `status`:

Status	Meaning	What the UI should do
400	Invalid/expired token (verify/reset)	"This link is invalid or expired."
401	Not logged in / session expired	Redirect to <code>/login</code> .
403	Email not verified (on login)	"Please verify your email first." Offer to resend later.
409	Email already registered	"That email is already in use." → link to login.
422	Bad input (e.g. not 11 numbers)	"Check your inputs."
429	Rate limited (too many AI calls)	"Slow down a moment and try again."
502	Prediction service error (HF down/cold)	"The model is waking up, please retry." + retry.

Example:

```
js
```



```
try {
  await api("/auth/login", { method: "POST", body: { email, password } });
} catch (e) {
  if (e.status === 403) showError("Please verify your email first.");
  else if (e.status === 401) showError("Wrong email or password.");
  else showError("Something went wrong, try again.");
}
```

9. Cold starts & loading states (important for UX)

The backend runs on Render's **free tier**, which **sleeps after ~15 min of inactivity**, and the model Space also cold-starts. So:

- The **first request after idle can take 30–60 seconds** (both the backend and the model waking up). Subsequent requests are fast.
- Always show a **loading state**, and consider a "**waking up the model...**" message + a **retry** on `(502)/timeout` for AI calls.
- Don't treat a slow first response as a failure — give it up to ~60s before erroring.

```
js

async function predictWithRetry(data, tries = 2) {
  for (let i = 0; i < tries; i++) {
    try { return await api("/api/predict", { method: "POST", body: { data } }); }
    catch (e) { if (e.status === 502 && i < tries - 1) continue; throw e; }
  }
}
```

10. Common pitfalls (save yourself the debugging)

1. **Forgetting** `(credentials: 'include')` → login seems to work, but `(/auth/me)` returns 401 and the user "isn't logged in". This is the classic one. Use the `(api())` helper everywhere.
2. **CORS error before anything works** → your exact origin isn't in the backend allowlist yet. Send me your URL(s) so I add them to `(FRONTEND_ORIGIN)`.
3. **Calling** `(/auth/google)` **with** `(fetch())` → it won't work. It must be a full-page `(window.location.href)` navigation.
4. **Doing auth calls from Next.js server components / SSR** → the user's HttpOnly cookie lives in the **browser**, so run session-dependent calls **client-side** (client components / effects). SSR `(fetch)` won't carry the user's cookie unless you manually forward it.

5. **Trying to read the session cookie in JS** → it's HttpOnly, you can't, and you don't need to. Use `/auth/me`.
 6. **Relying on `result[1]` for the tier** → use `result[0].label` instead (see §7).
 7. **Scaling biomarkers on the client** → don't. Send raw values; the server scales.
-

11. Quick reference

Base URL: `https://cardiotox-backend.onrender.com`

```
POST /auth/register      {email, password, display_name}  -> 201
GET  /auth/verify?token=... (email link; backend redirect)
POST /auth/login         {email, password}                  -> 200 +
cookie
POST /auth/logout                          -> 200/204
POST /auth/password/forgot {email}                      -> 200
(always)
POST /auth/password/reset {token, new_password}          -> 200
GET  /auth/me                          -> 200
profile / 401
GET  /auth/google          (full-page redirect, not fetch)
POST /api/predict          {data:[11 raw numbers]}          ->
[labelObj, tierString]
POST /api/explain          {data:[11 raw numbers]}          -> [shapObj]
GET  /healthz              -> health
check
```

Golden rules: `credentials: 'include'` on everything · no tokens/localStorage · `/auth/me` to check login · Google = redirect · 11 raw biomarkers in order · handle 401/403/409/422/429/502 · expect a slow first request.

12. Not built yet (so you know the current limits)

- **User roles / admin vs normal users** — not implemented yet. Right now every authenticated user is equal. Role-based access is planned; I'll extend `/auth/me` with a `role` field when it lands, so build your UI to tolerate that field appearing later.
- **Email delivery** is currently limited to a test address until a sending domain is verified, so during integration, verification/reset emails may only reach my test inbox. Ping me if you need a verified test account to log in with.

Questions or something returning a status you don't expect? Send me the request + the status code and I'll check the backend logs (every request has an `x-request-id` we can trace).

